

1 Develop a Flask form application that stores user input into an SQLite database.

Aim

To create a Flask web application with a form that collects user input (like name and email) and stores it into an SQLite database, then displays the stored data.

Algorithm

1. Initialize Flask app
 - Import Flask, SQLite, and required modules.
2. Create SQLite database
 - Define a table with fields: , , .
3. Design HTML form
 - Simple form with and fields.
4. Handle form submission
 - On , capture form data and insert into database.
5. Display stored data
 - Create a route to fetch and show all entries.

CODE

```
from flask import Flask, request, redirect
import sqlite3
```

```
app = Flask(__name__)
```

```
# Step 1: Initialize database
```

```
def init_db():
```

```
    conn = sqlite3.connect('users.db')
```

```
    cursor = conn.cursor()
```

```
    cursor.execute("""
```

```
CREATE TABLE IF NOT EXISTS users (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    name TEXT NOT NULL,  
    email TEXT NOT NULL  
)  
""  
conn.commit()  
conn.close()
```

```
init_db()
```

```
# Step 2: Form route
```

```
@app.route('/', methods=['GET', 'POST'])
```

```
def index():
```

```
    if request.method == 'POST':
```

```
        name = request.form['name']
```

```
        email = request.form['email']
```

```
    # Insert into database
```

```
    conn = sqlite3.connect('users.db')
```

```
    cursor = conn.cursor()
```

```
    cursor.execute("INSERT INTO users (name, email) VALUES (?, ?)", (name,  
email))
```

```
    conn.commit()
```

```
    conn.close()
```

```
    return redirect('/users')
```

```
return ""
```

```
<h2>User Form</h2>

<form method="POST">

    Name: <input type="text" name="name"><br>

    Email: <input type="text" name="email"><br>

    <input type="submit" value="Submit">

</form>

'''
```

Step 3: Display stored users

```
@app.route('/users')
def users():

    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM users")
    rows = cursor.fetchall()
    conn.close()

    output = "<h2>Stored Users</h2><ul>"

    for row in rows:

        output += f"<li>{row[1]} - {row[2]}</li>"

    output += "</ul>"

    return output

if __name__ == '__main__':
    app.run(debug=True)
```

OUTPUT

❑ Run the app:

```
python app.py
```

Open browser at <http://127.0.0.1:5000/>.

3. Fill form:

Name: Yogin

Email: yogin@example.com

☐ After submission, visit /users →

Stored Users

- Yogin - yogin@example.com

☒ Conclusion

- The Flask app successfully stores user input into an **SQLite database**.
- The /users route displays all saved entries.
- This is a **basic CRUD foundation** (Create + Read).

2 Write a program to display stored records from SQLite using Flask.

Aim

To write a Flask application that connects to an SQLite database and displays all stored records (e.g., user details) on a web page.

Algorithm

1. Initialize Flask app
 - Import Flask and SQLite modules.
2. Connect to SQLite database
 - Open connection and fetch records from a table (e.g.,).
3. Create route
 - Query all rows from the table.
 - Display them in HTML format.
4. Run the app
 - Start Flask server and view records in browser.

CODE

```
from flask import Flask
import sqlite3

app = Flask(__name__)

# Route to display stored records
@app.route('/users')
def users():
    # Connect to SQLite database
    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()

    # Fetch all records from 'users' table
    cursor.execute("SELECT * FROM users")
    rows = cursor.fetchall()

    conn.close()

    # Build HTML output
    output = "<h2>Stored Users</h2><ul>"

    for row in rows:
        output += f"<li>ID: {row[0]} | Name: {row[1]} | Email: {row[2]}</li>"

    output += "</ul>"

    return output

if __name__ == '__main__':
```

```
app.run(debug=True)
```

OUTPUT

Stored Users

- ID: 1 | Name: Yogin | Email: yogin@example.com
- ID: 2 | Name: Alex | Email: alex@example.com

✓ Conclusion

- This Flask program connects to SQLite and retrieves all records from the table.
- The records are displayed neatly in the browser.
- This forms the Read part of CRUD operations.